

Příklady na dědičnost, polymorfismus a abstrakci:

1. Škola

První část (dědičnost, polymorfismus):

- Vytvořte třídu *Člověk*, která má atributy *jméno* (veřejný) a *rok_narození* (soukromý).
- Přidejte třídě *Člověk* getter pro atribut *rok_narození*.
- Vytvořte třídu *Škola*, která obsahuje atribut *název*.
- Vytvořte třídu *Žák*, která dědí od třídy *Člověk* a přidává veřejný atribut *navštěvovaná_škola* a chráněný atribut *splněné_předměty* (množina řetězců, výchozí hodnota prázdná množina).
- Přidejte třídě *Žák* veřejnou metodu *splnit_předmět*, která jako parametr přebírá zkratku předmětu a daný předmět přidá žákovi do množiny *splněných_předmětů*.
- Vytvořte třídu *Student*, která dědí ze třídy *Žák* a přidává soukromý atribut *počet_kreditů* (celé číslo, výchozí hodnota 0).
- Přidejte třídě *Student* getter pro atribut *počet_kreditů*.
- Překryjte v třídě *Student* metodu *splnit_předmět* tak, aby navíc přebírala parametr *kredity* (celé číslo) se základní hodnotou 4. Hodnotu z parametru *kredity* přičte do atributu *počet_kreditů* a zavolá původní implementaci metody z rodičovské třídy.
 - Výchozí hodnota u parametrů se uvádí za znak *rovná se*. (*parametr: typ = hodnota*)
 - Na rodičovské vlastnosti se přistupuje pomocí zabudované metody *super*.

Druhá část (statické vlastnosti):

- Třídě *Žák* přidejte property *předměty*, která vrátí *splněné_předměty* jako seznam (ne jako množinu).
- Třídě *Žák* přidejte statický atribut *počet_studujících* (celé číslo, výchozí hodnota 0). V konstruktoru žáka tuto hodnotu zvýšte o 1.
 - Jaký přístupový kvalifikátor se atributu hodí (veřejný/chráněný/soukromý)? Vyberte a naimplementujte.
 - Přidejte statickou metodu pro čtení tohoto atributu. (statický getter/property nelze)
 - Jak se chová atribut při vytváření žáků a studentů? Do komentáře popište tuto situaci.
- Přidejte třídě *Student* statický soukromý atribut *aktivní_předměty* (množina řetězců, výchozí hodnota prázdná množina).
- Přidejte třídě *Student* statickou veřejnou metodu *přidat_předmět*, která jako parametr přebírá zkratku předmětu a přidá ji do množiny *aktivních_předmětů*.
- Upravte ve třídě *Student* implementaci metody *splnit_předmět* tak, aby kontrolovala, že uvedená zkratka předmětu je v množině *aktivních_předmětů*. Pokud není, metoda vypíše chybovou hlášku.

Bonus:

- Ve třídě *Student* přidejte metodu *smazat_předmět* analogicky k metodě *přidat_předmět*.
- Překryjte ve třídě *Student* property *předměty* tak, aby nevracela předměty neobsažené v *aktivních_předmětech*.

2. Skleník

Rostlina má „dny od zasazení“, což odpovídá počtu dnů růstu rostliny. Po „růst rostliny“ dnech je rostlina vyrostlá. Po „růst plody“ dnech vyrostou rostlině plody. Po „růst shnití“ rostlina shnije.

Časová osa: **[vytvoření | “růst rostliny” | “růst plody” | “růst shnití” | konec]**

1. Vytvořte abstraktní třídu *Rostlina*.
 - a. Atributy zadané z parametru:
 - i. *růst_rostliny* (int, soukromý)
 - b. Atributy s implicitní hodnotou:
 - i. *dny_od_zasazení* (int, soukromý, hodnota 0)
 - ii. *je_sklizená* (bool, soukromý, hodnota *False*)
2. Přidejte třídě abstraktní metody (popř. lepší použít properties – viz dokumentace):
 - a. *násobek_růstu_plodů* (bez parametrů, vrací float) – násobek hodnoty atributu *růst_rostliny* a hodnoty této metody určuje počet dnů potřebných na růst plodů.
 - i. Např. *růst_rostliny* = 5, *násobek_růstu_plodů* = 1.6 -> plody vyrostou po 8 dnech od zasazení rostliny.
 - b. *násobek_shnití* (bez parametrů, vrací float) – podobně jako předchozí, ale určuje počet dnů do shnití rostliny.
3. Přidejte třídě property *je_vyrostlá*, která vrací logickou hodnotu, zda rostlina již rostla dostatečný počet dnů, aby byla plně vyrostlá ale ne ještě shnilá.
4. Přidejte třídě property *má_plody*, která vrací logickou hodnotu, zda rostlina rostla dostatečný počet dnů na to, aby měla plody. Rostlina ale nesmí být sklizená.
5. Přidejte třídě metodu *sklidit*, která vrací logickou hodnotu, zda se sklizeň podařila. Kontroluje, zda má rostlina plody, ale nebyla zatím sklizená. Pokud lze sklidit, nastaví atribut *sklizená* na *True* a vrátí *True*. Jinak vrací *False*.
6. Přidejte třídě metodu *vyrůst_o_den*, která zvýší *dny_od_zasazení* o 1.
7. Přidejte alespoň dvě třídy různých druhů rostlin (např. rajčata, jahody), které dědí od třídy *Rostlina*. Naimplementujte jim abstraktní metody.
8. Naimplementujte dceřiným třídám metodu *__str__*:
 - a. Formát: „[8/15] – roste“
 - b. *[dny_od_zasazení/růst_rostliny]* – stav rostliny (roste/s plody/sklizená/shnilá)
9. V *main.py* vytvořte seznam rostlin. Přidejte do něj dohromady alespoň 5 instancí vašich rostlin.
10. Vytvořte cyklus, který bude simulovat postup času. Každá iterace cyklu bude představovat jeden den. Každý den nechte povyrůst všechny rostliny v seznamu a vypište pod sebe jejich stavy. Jakmile lze rostliny sklidit, sklidte je. Cyklus ukončete, když všechny rostliny shnijí.

Bonus:

1. Přidejte náhodnou šanci, že na rostlinu zaútočí škůdci a tím zpomalí její růst.